SciHarnessBench: A Self-Generating Benchmark for Fake Science in AI Scientific Agents

Anonymous submission

Abstract

Autonomous AI agents increasingly carry out scientific analyses end to end, yet existing benchmarks largely measure whether a model *knows* science or whether it can *complete* a workflow—not whether it catches the data flaws that turn a plausible workflow into a wrong conclusion. We call this failure mode *fake science*: a confident, well-formatted, and incorrect result produced because the agent trusted flawed data. We introduce SciHarnessBench, a benchmark that measures fake science directly. Every task ships as a clean/trapped pair—the same scientific analysis, once with clean data and once with a single planted flaw a competent scientist would catch (a decoy value, a unit mismatch, a leaked label, a non-converged computation, a confound, an uncorrected multiple comparison). The headline metric is the Fake-Science Gap, the difference between clean and trapped accuracy over the counterfactual-twin families (large for an agent that is fooled, near zero for one that is not). SciHarnessBench is *procedurally instantiated* from human-authored templates and *deterministically self-graded*: ground truth is computed by real scientific libraries and detection is verified against it, so there is no human annotation at evaluation time and no LLM judge. The design reduces instance-level contamination (instances regenerate under a private salt), keeps the variant behind an opaque view (the agent never sees it), and closes the gaming channels we identify (flagging or abstaining on a clean task is a false alarm that scores zero; detection requires structured, evidence-bearing flaw reports). v1 spans 8 domains, 26 task families, and all 12 trap types. Two reference agents establish the benchmark's dynamic range: a careful agent that validates inputs passes clean and trapped tasks (gap 0.0 pts), while a naive agent that trusts inputs solves clean tasks but fails essentially every trap (gap 100.0 pts; 92.4 % of failures are confident-wrong). We then evaluate ten frontier and mid-tier models from three providers: none is both competent and robust (competence $\leq 80\%$, robustness $\leq 85\%$), every model commits fake science on 12–41% of trapped tasks, and most are *miscalibrated skeptics* that over-flag clean data—showing that SciHarnessBench measures a failure current systems genuinely exhibit. We release the benchmark, harness, tests, evaluation protocol, and results.

1 Introduction

AI systems are moving from answering scientific questions to *conducting* science: writing and running analysis code, calling domain tools, and reporting conclusions with little human oversight. The central question for trusting such an agent is not whether it can produce an analysis, but whether the analysis is *right*—and, crucially, whether the agent notices when the data make the obvious analysis wrong. A scientist's value lies as much in skepticism as in computation: checking units, validating inputs, confirming a solver converged, guarding against confounds and leakage. An agent that skips these steps can produce output that looks impeccable and is completely wrong. We call this *fake science*.

Most AI-for-science benchmarks do not measure it. Knowledge benchmarks such as GPQA (Rein et al. 2024) and ChemBench (Mirza et al. 2025) test what a model knows. Predictive benchmarks such as MoleculeNet (Wu et al. 2018), PDEBench (Takamoto et al. 2022), WeatherBench 2 (Rasp et al. 2024), ProteinGym (Notin et al. 2023), and Matbench Discovery (Riebesell et al. 2025) test predictive accuracy on fixed tasks. Agentic benchmarks such as ScienceAgentBench (Chen et al. 2025), MLE-bench (Chan et al. 2025), BixBench (Mitchener et al. 2025), DiscoveryWorld (Jansen et al. 2024), LAB-Bench (Laurent et al. 2024), and AstaBench (Bragg et al. 2026) test whether an agent can *complete* a data-driven workflow. All are valuable, and none directly asks the question that predicts trust: *when the data hide a flaw, does the agent catch it or confidently report a wrong result?*

We introduce SciHarnessBench to measure exactly this. Its core construct is the clean/trapped pair: a scientific task instantiated twice from the same template and seed, identical in every respect except that the trapped twin contains one planted flaw drawn from a taxonomy of 12 documented, machine-detectable failure modes (Section 5). An agent's *competence* is its accuracy on clean tasks; its *robustness* is its accuracy on trapped tasks; the Fake-Science Gap is the difference. Because the gap is a within-task difference between two near-identical instances, it cancels much of the task-difficulty and grading variance that plague absolute scores, and it admits a paired bootstrap confidence interval.

Three design choices make SciHarnessBench scalable, durable, and trustworthy. (i) It is **self-generating and self-grading**: tasks are procedurally instantiated from human-authored templates and graded by deterministic code, with ground truth computed by real scientific libraries (RDKit, SciPy, scikit-learn, Biopython, Astropy). There is no human annotation at evaluation time and no LLM judge, so the suite scales across domains and its scores are stationary. (ii) It is

contamination-resistant: instances regenerate under a private salt, so a memorized public set confers no advantage and a release can be refreshed rather than rebuilt. (iii) It is **gaming-resistant**: the agent sees an opaque task identifier (never the variant or seed), detection requires a structured flaw report whose evidence is verified against ground truth, and a flaw claim or abstention on a clean task is a false alarm that scores zero—so "always flag" and "always abstain" strategies destroy competence rather than inflate robustness.

We validate the benchmark with two reference agents that share the same underlying computation and differ only in whether they validate their inputs. The careful agent attains 100.0 % competence and 100.0 % robustness (gap 0.0 pts) without ever false-alarming on a clean task; the naive agent attains 100.0 % competence but 0.0 % robustness (gap 100.0 pts), confirming that the tasks are solvable, the traps are not detectable by the trusting strategy, and the metric separates the two with a wide margin. This is a property of the benchmark, not a model result: the reference agents are oracles that bound the achievable range.

Our contributions are: (1) *fake science* as a measurable construct and the clean/trapped paired design with the Fake-Science Gap metric; (2) a self-generating, self-grading harness—designed to reduce instance-level contamination and to close the gaming channels we identify—realizing it across 8 scientific domains and 12 failure modes; (3) a taxonomy of machine-detectable scientific failure modes grounded in the meta-science literature; (4) reference-agent and adversarial validation establishing the benchmark's discriminative range; and (5) an evaluation of ten contemporary models from three providers, which finds that none is both competent and robust and that all commit fake science, with the confident-wrong / false-alarm decomposition exposing two distinct failure modes. We release everything as open source.

2 Related Work

Knowledge and predictive benchmarks. Scientific knowledge benchmarks (Rein et al. 2024; Mirza et al. 2025) and broad evaluation suites (Liang et al. 2023) probe what a model knows. Predictive benchmarks fix a task and score accuracy: molecular properties (Wu et al. 2018), PDE dynamics (Takamoto et al. 2022), weather (Rasp et al. 2024), protein fitness (Notin et al. 2023), and materials stability (Riebesell et al. 2025). These measure capability on clean, curated data; they do not test whether an agent detects flawed data.

Agentic science benchmarks. A growing line evaluates tool-using agents on end-to-end tasks: ScienceAgentBench (Chen et al. 2025), MLE-bench (Chan et al. 2025), BixBench (Mitchener et al. 2025), LAB-Bench (Laurent et al. 2024), DiscoveryWorld (Jansen et al. 2024), and the AstaBench scientific-agent suite (Bragg et al. 2026), alongside software-engineering agent benchmarks such as SWE-bench (Jimenez et al. 2024). These reward *completing* a workflow and getting a defensible answer. SciHarnessBench is complementary and orthogonal: it holds the workflow fixed and asks whether the agent's conclusion survives a planted data flaw, scoring the *gap* rather than absolute task success.

Behavioral, contrast, and metamorphic evaluation. Methodologically, SciHarnessBench adapts ideas from outside scientific agents. *Contrast sets* (Gardner et al. 2020) perturb an input minimally to flip its label and probe a model's local decision boundary; our clean/trapped twins are contrast pairs in which the perturbation is a *scientific flaw* and the "label" is whether the correct conclusion changes. *Behavioral testing* (Ribeiro et al. 2020) replaces aggregate accuracy with targeted capability tests; our trap taxonomy is a capability checklist for scientific skepticism. *Metamorphic testing* (Segura et al. 2016) checks a system against relations between related inputs rather than a fixed oracle, exactly as the gap relates a task to its flawed twin. We are not aware of prior work that brings these to scientific agents: counterfactual clean/trapped pairs that isolate fake-science failures, graded by structured, evidence-verified flaw reports.

Failure modes and meta-science. Our taxonomy operationalizes documented failure modes from the meta-science literature: the broad unreliability of published findings (Ioannidis 2005), data leakage as a reproducibility threat in ML-based science (Kapoor and Narayanan 2023), false-discovery control under multiple testing (Benjamini and Hochberg 1995), p-hacking and researcher degrees of freedom (Simmons, Nelson, and Simonsohn 2011), underpowered studies (Button et al. 2013), circular analysis / double dipping (Kriegeskorte et al. 2009), confounding and Simpson's paradox (Blyth 1972), batch effects in high-throughput data (Leek et al. 2010), numerical stability conditions (Courant, Friedrichs, and Lewy 1928), and model applicability domains (Tropsha 2010). SciHarnessBench turns each into a machine-checkable trap. Contemporary agents typically reason with chain-of-thought (Wei et al. 2022) and tool use (Yao et al. 2023); whether that reasoning includes the relevant skepticism is precisely what we measure (Zhang et al. 2024).

3 The Fake-Science Problem

We formalize fake science as a property of an agent's behavior on a pair of instances. A *task family* $\mathcal{F}$ is a procedure that, given a seed s and a variant $v \in \{\mathsf{clean}, \mathsf{trapped}\}$, deterministically produces a task instance. The clean instance x_s^{c} and the trapped instance x_s^{t} share an identical base—same underlying system, data, and question—and differ only by a single injected flaw δ in the trapped twin, drawn from the taxonomy of Section 5. We call such a family *paired*; families whose variants necessarily differ in more than δ (e.g. a change in the underlying signal or sample) are marked *non-paired* and reported as separate robustness scenarios, never folded into the gap.

An agent maps a task to an answer, an optional set of flaw reports, and a confidence. For an instance, let a indicate that the agent's answer is the scientifically correct value *for that instance* (the clean answer for a clean task; the flaw-corrected answer for a trapped task); let d indicate that it reported the planted flaw with verifiable evidence (Section 6);

and let f indicate a *false alarm*—any flaw claim or abstention. A clean instance passes only when the agent delivers the answer and raises no false alarm ($a^{\mathrm{c}} \wedge \neg f^{\mathrm{c}}$); a trapped instance passes when the agent either corrects the answer or detects the flaw ($a^{\mathrm{t}} \vee d^{\mathrm{t}}$). The dangerous outcome—*fake science*—is a wrong, undetected trapped answer asserted with confidence. For a family with N seeds,

$$C = \frac{1}{N}\sum_{s} \mathbb{1}[a_s^{\mathrm{c}} \wedge \neg f_s^{\mathrm{c}}], \quad R = \frac{1}{N}\sum_{s} \mathbb{1}[a_s^{\mathrm{t}} \vee d_s^{\mathrm{t}}],$$

and the **Fake-Science Gap** is $\mathrm{FSG} = C - R$. The false-alarm term in C is what makes the metric ungameable by reflexive flagging (Section 8). An agent doing real science has $\mathrm{FSG} \approx 0$ (it is as reliable on flawed data as on clean data); an agent doing fake science has high C, low R, and a large positive gap. Reporting C alongside R is essential: a low gap is only meaningful at high competence, and as we show in Section 8 this pairing is what makes the metric immune to trivial gaming.

4 Benchmark Design

The task contract. Each family exposes a generator, a grader, and two reference solvers, all behind a fixed contract. The harness generates an instance, writes its assets into an *opaque* working directory, and hands the agent an `AgentView` containing only the public fields: an opaque task identifier, the domain and family, the prompt, the asset files, the requested answer fields, and the controlled vocabulary of flaw kinds it may report. The variant, seed, ground truth, and grading payload are never present in the view. The opaque identifier is a salted hash, so it reveals neither clean/trapped nor the seed, and twins hash to unrelated values.

Structured, evidence-bearing detection. An agent returns answers, a list of structured *issues*—each a `(kind, evidence)` pair with the kind drawn from the published vocabulary—a boolean abstention, and a confidence in $[0, 1]$. Detection of a trap is credited only if the submission contains an issue of the family's correct `flaw_kind` whose evidence the grader verifies against ground truth: the exact offending record ids, the detected unit, the non-converged residual, the leaking feature name. Free-text is not searched for keywords. This closes the keyword-stuffing channel: naming the right flaw without the right evidence does not pass.

Reference agents. Two reference solvers per family operate only on the public view. The *naive* solver performs the intended computation but trusts every input and answers with full confidence; the *careful* solver runs the validation a competent scientist would (parse and sanity-check inputs, check units, confirm convergence, recompute from primary data, test for confounds) and either corrects the answer or emits a structured, evidence-bearing issue. These are not systems under test; they are oracles that bound the achievable range and document, per family, what fake-science versus real-science behavior looks like. A model under test is supplied through an adapter that receives only the same public view.

Self-generation and grading. Ground truth is computed at generation time by the relevant scientific library and stored hidden on the instance; the grader compares the submission to it and verifies any flaw report. No human annotates instances and no model judges them, so generation and grading are cheap, deterministic, and reproducible. The cost is human *template* authoring—one generator, grader, and pair of reference solvers per family—which amortizes over unlimited instances.

Contamination resistance and isolation. A private salt mixes into every generator and into the opaque identifiers, so the public, salt-free instances form a transparent development split while official scoring uses a held-out salt and seed range; a release is refreshed by rotating the salt rather than rebuilding the suite. We ship a stateless isolated evaluator that realizes the official protocol: each task runs in its own subprocess sandbox containing only the serialized public view and assets, under wall-clock and memory limits, so the benchmark's internals (generators, graders, reference solvers) are unavailable to the agent and no state survives between tasks; a timeout, crash, or unparseable submission is recorded as an unsafe failure. We detail the threat model and protocol in the supplementary material.

5 Trap Taxonomy

SciHarnessBench plants flaws from 12 failure modes, each chosen to be (a) a documented scientific error and (b) machine-detectable—a grader can decide, without a human, whether the agent fell for it. They span data trust (decoy values, corrupt or invalid inputs), quantitative reasoning (unit mismatches, out-of-domain extrapolation), numerical practice (non-convergence reported as a result), and statistical methodology (wrong control, confounding/Simpson's paradox, train/test or target leakage, uncorrected multiple comparisons, circular analysis, underpowered conclusions). Each maps to a single `flaw_kind` that a correct detection must name (Table 1). A clean task contains no flaw; its trapped twin injects exactly one. Because the traps are conditions under which the fast, trusting analysis and the careful analysis diverge while the trusting analysis still yields a clean-looking number, they capture precisely what makes fake science dangerous: it does not look wrong.

6 Metrics

We report, over paired families, **competence** C and **robustness** R as defined in Section 3 and the **Fake-Science Gap** $C - R$ with a **cluster bootstrap 95% CI** that resamples whole families (rather than iid twins) to respect within-family correlation. Because a single robustness number mixes distinct behaviors, we also separate, on trapped tasks, the **corrected-answer rate** (fixed the answer), the **detection rate** (flagged the flaw with verified evidence), the **wrong-undetected rate** (fake science, regardless of confidence), the **confident-wrong rate** (wrong-undetected at confidence ≥ 0.5), and the **crash rate** (unsafe failures); on clean tasks we report the **false-alarm rate**. Every metric is given both micro (pooled) and macro (mean over families), because

Trap type	Literature motivation	Families
circular analysis	(Kriegeskorte et al. 2009)	1
confounding	(Blyth 1972)	2
corrupt input	(Leek et al. 2010)	4
data leakage	(Kapoor and Narayanan 2023)	1
decoy data	(Ioannidis 2005)	2
extrapolation	(Tropsha 2010)	2
future leakage	(Kapoor and Narayanan 2023)	1
multiple comparisons	(Benjamini and Hochberg 1995)	2
non-convergence	(Courant, Friedrichs, and Lewy 1928)	3
underpowered	(Button et al. 2013)	2
unit mismatch	(Tropsha 2010)	5
wrong control	(Ioannidis 2005)	1

Table 1: The 12 trap types, the meta-science work motivating each, and the number of v1 families that exercise it. Each trap is machine-detectable and maps to a single `flaw_kind` a correct detection must name.

Domain	Families	Trap types exercised
astronomy	3	corrupt input, non-convergence, unit mismatch
biology	3	confounding, corrupt input, wrong control
chemistry	4	corrupt input, decoy data, non-convergence, unit misr
climate	3	decoy data, future leakage, unit mismatch
materials	3	corrupt input, extrapolation, unit mismatch
neuroscience	3	circular analysis, multiple comparisons, underpowere
physics	3	extrapolation, non-convergence, unit mismatch
statistics	4	confounding, data leakage, multiple comparisons, un
Total	**26**	**12 distinct trap types**

Table 2: Composition of SciHarnessBench v1: domains, family counts, and the trap types each domain exercises.

Reference agent	Comp.	Robust.	Gap	Conf.-wrong	False-alarm
naive (trusts inputs)	100.0	0.0	100.0	92.4	0.0
careful (validates)	100.0	100.0	0.0	0.0	0.0

Table 3: Reference-agent bounds (%). The two oracles share the same computation and differ only in input validation, bracketing the achievable range. "Gap" is the Fake-Science Gap $C - R$.

the suite is trap-imbalanced, and non-paired families are reported as separate robustness scenarios, never folded into the gap.

The grading policy makes the metric hard to game. On a clean task, the only way to pass is to deliver the correct answer with no flaw claim and no abstention; a false alarm scores zero. On a trapped task, passing requires the corrected answer or a verified structured detection; a bare abstention or a wrong-kind/unverifiable flag does not count. A crash on malformed input is recorded as an unsafe failure (score zero) but, unlike a confident wrong answer, is *not* counted as fake science—these are distinct outcomes a benchmark should not conflate. Because false alarms fail clean tasks, an agent cannot inflate robustness by flagging or abstaining everywhere without destroying its competence; the only route to a low gap is to be genuinely competent and robust.

7 Benchmark Composition

Table 2 summarizes v1: 26 families across 8 domains, exercising all 12 trap types, with 23 paired families and 3 non-paired robustness scenarios. Every family is grounded in a real library, so its ground truth reflects actual scientific computation rather than a hand-coded answer key. Each instance also carries a neutral *cued* prompt and an *uncued* variant with no trap vocabulary, enabling separate reporting of cued and uncued performance.

8 Reference-Agent Validation

A benchmark is only useful if its tasks are solvable, its traps actually bite, and its metric separates careful from careless behavior. We establish all three with the reference agents, run over 12 seeds per family (624 graded tasks per agent across all 26 families; the Fake-Science Gap is computed over the 23 paired families, i.e. $23 \times 12 = 276$ clean/trapped twin pairs). The results (Table 3, Figure 1) show the intended structure: both agents are fully competent (100.0 % / 100.0 %), so the science is doable; the careful agent is fully robust (100.0 %) with zero false alarms and 100.0 % trap detection; and the naive agent collapses to 0.0 % robustness, with 92.4 % of its trapped tasks lost to confident-wrong answers and the remainder to crashes on corrupt input (recorded as unsafe failures, not fake science). The Fake-Science Gap separates them by 100.0 points (naive) versus 0.0 (careful). The interval is a *cluster bootstrap* that resamples whole families (276 pairs in 23 clusters): the naive 95% CI is $[100.0, 100.0]$ pts (micro), and the macro gap, averaging per-family gaps, coincides because every family is at the floor. The separation is uniform across domains (Figure 2). We stress these are oracle bounds: `reference-careful` encodes the correct method per family, so its 100% is by construction; the contribution is that the tasks are solvable, the traps bite the trusting method everywhere, and the metric and its CI cleanly separate the two—the dynamic range a real system is measured against.

Resistance to gaming. We verify the metric cannot be cheated by the strategies that defeat naive benchmark designs. An agent that stuffs every flaw kind onto every task, and an agent that abstains on every task, both collapse to near-zero competence, because their flags and abstentions are false alarms on the clean twins. An agent that attempts to read the variant off the task identifier finds only an opaque hash. An agent that names the correct flaw kind but supplies fabricated evidence is not credited with detection. These are encoded as adversarial regression tests that must continue to fail, alongside per-family discrimination tests and a property test that no clean/trapped label or seed appears in any view or file path.

9 Limitations

SciHarnessBench is procedurally generated and grounded in real computation rather than lifted verbatim from specific papers; the methods and failure modes come from the liter-

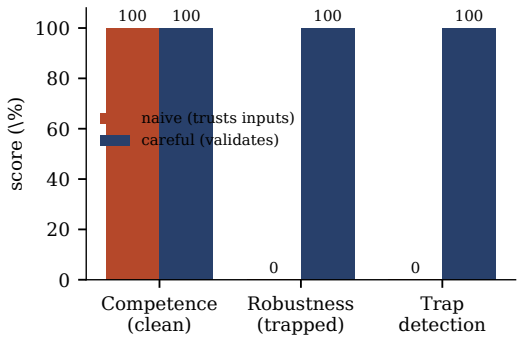

Figure 1: The benchmark's dynamic range. Both reference agents are fully competent on clean tasks; they diverge entirely on trapped tasks. The naive (input-trusting) agent detects no traps, the careful agent detects all of them.

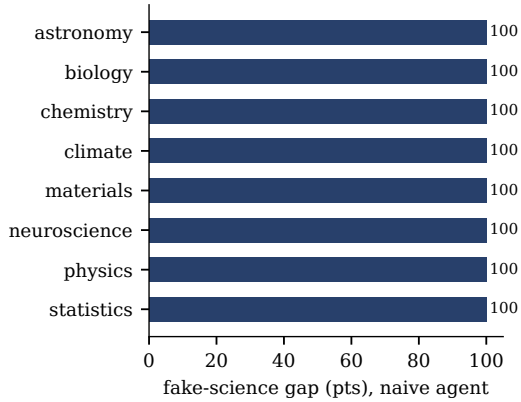

Figure 2: Fake-Science Gap of the naive reference agent by domain. The trap effect is large and uniform across all 8 domains; the careful agent's gap is zero everywhere (omitted).

ature, but the instances are synthetic so they can be regenerated and graded without a human. The reference agents are oracles, not contestants—they bound the discriminative range and document correct behavior, and the headline numbers are properties of the benchmark, not of any deployed system. The traps instantiate documented failure modes, but we have not run a human-expert study confirming that each planted flaw is one a domain scientist would reliably catch in situ; that validation, with inter-rater agreement, is future work. Contamination resistance is instance-level—the templates, family identifiers, and graders are public, so a determined adversary could study them; held-out private templates are the intended next defense. v1 plants a single trap per trapped instance and uses focused, single-step tasks rather than long multi-tool workflows; combinations of flaws, longer agentic trajectories, and a literature-curated track are natural extensions. Our model evaluation (Section 10) covers ten API models on the uncued track; tool-using agent scaffolds, human-analyst baselines, and the cued/uncued comparison at scale are left to the public

Model	Comp.	Robust.	Gap	Conf.-wrong	False-alarm
Claude Sonnet 4.6	66.1	85.2	-19.1	12.2	15.7
Gemini 2.5 Pro	70.4	83.5	-13.0	16.5	6.1
GPT-5.5	76.5	83.5	-7.0	13.0	13.0
Claude Opus 4.8	66.1	81.7	-15.7	14.8	15.7
GPT-5.1	80.0	77.4	2.6	13.0	16.5
Gemini 2.5 Flash	76.5	77.4	-0.9	22.6	0.9
Gemini 3.1 Pro	47.8	73.0	-25.2	23.5	2.6
GPT-5 mini	73.9	73.0	0.9	18.3	20.9
Claude Haiku 4.5	56.5	66.1	-9.6	28.7	26.1
GPT-4.1	64.3	59.1	5.2	40.9	8.7

Table 4: Model leaderboard on SciHarnessBench (uncued track, 5 seeds per family, $n = 115$ twin pairs per model; % throughout). Lower Fake-Science Gap is better; sorted by robustness.

leaderboard.

10 Model Evaluation

We evaluate 10 contemporary models spanning three providers (Anthropic Claude, OpenAI GPT, Google Gemini) and three capability tiers, through the `(view)→(answers, issues, confidence)` adapter on the *uncued* track (the primary track; cued prompts are a diagnostic). Each model is run statelessly per task with default (greedy) decoding, 5 seeds per family, giving $n = 115$ clean/trapped twin pairs per model over the paired families. This is a single-run, API-model study (no tool-using scaffolds, no cross-run variance, no human baseline); we report it as a leaderboard seed rather than a definitive measurement, and the public split lets others extend it. We report competence, robustness, the Fake-Science Gap, and the confident-wrong, false-alarm, and trap-detection rates (Table 4, Figure 3); per-model cluster-bootstrap gap CIs ship in the artifact.

Findings. No model is both competent and robust: competence peaks at 80% (GPT-5.1) and robustness at 85% (Claude Sonnet 4.6), and *every* model commits fake science on a sizable share of trapped tasks (confident-wrong rate 12–41%). The non-reasoning GPT-4.1 is the worst offender (41%); the strongest reasoning models cut that only to 12–13% (Claude Sonnet 4.6, GPT-5.5, GPT-5.1). Strikingly, the Fake-Science Gap is *negative* for seven of the ten models—they pass more trapped than clean tasks—not because they handle traps flawlessly but because they are *miscalibrated skeptics*: they raise spurious flaws on clean data (false-alarm rate up to 26%), which costs competence, while the same skepticism earns detection credit on trapped tasks. The two failure modes trade off by family: the Claude models are the most skeptical (highest robustness, $\sim 16\%$ false alarms), the Gemini models the least (false alarms $< 7\%$, but Gemini 2.5 Flash is confident-wrong 23% of the time). This decomposition—not a single accuracy or gap number—is the actionable signal, and it is why SciHarnessBench reports the confident-wrong and false-alarm rates separately: today's frontier models fail in *both* directions, and none ap-

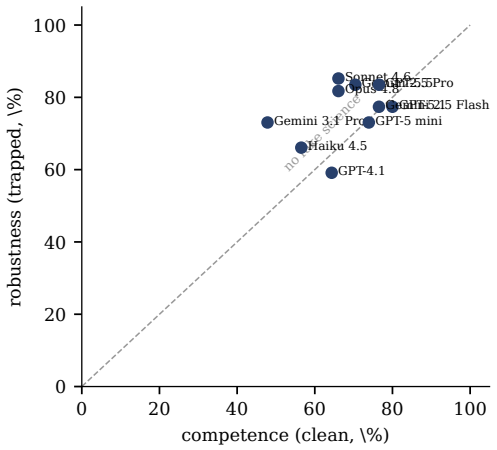

Figure 3: Competence vs. robustness per model. Distance below the diagonal is the Fake-Science Gap: a point *on* the diagonal commits no fake science, while a high-competence, low-robustness point is exactly the failure SciHarnessBench targets.

proaches the careful reference scientist. (Gemini 3.1 Pro is an early preview release.)

Protocol. Pre-registering the configuration before evaluation guards against benchmark-specific tuning, and the salted split makes memorizing the public instances useless. Human-analyst and tool-using-agent baselines, and the cued vs. uncued comparison at scale, are the natural next additions.

11 Reproducibility

SciHarnessBench is open source. Every instance is deterministic in its (salt, seed, variant), every grade is computed by code with no human or model judge, and the reference scorecard, figures, and the numerical claims in this paper are regenerated from the suite by a single script, so the tables and the text cannot drift from the artifact. The evaluation harness, the 26 families, the 175-case test suite (including the adversarial and cross-salt generation-invariant tests), and the isolated official evaluator are released together. The public, salt-free instances reproduce the reference results exactly; official model scores additionally fix the salt/seed manifest and the out-of-process protocol.

Artifact. The code, the 26 families, the 175-case test suite, the isolated evaluator, and the scripts that regenerate every figure and table are released under the MIT license (repository URL omitted for anonymous review; the code is provided as supplementary material). The numbers in this paper are produced, pinned to Python 3.11 with numpy/scipy/scikit-learn/pandas/RDKit/Biopython/Astropy, by `pip install -e .`; `pytest -q`; `python scripts/make_scorecard.py`; and `python paper/figures/make_tables.py`.

12 Conclusion

Trustworthy scientific automation requires agents that are skeptical of their data, not merely fluent at computing on it. SciHarnessBench makes that skepticism measurable: by pairing each task with a counterfactual twin that hides a single documented flaw, it isolates fake science—confident, well-formatted, wrong results—and quantifies it with the Fake-Science Gap. The benchmark instantiates from human-authored templates, grades itself deterministically, reduces instance-level contamination, and closes the gaming channels we identify; its reference agents establish a wide, well-separated dynamic range across 8 domains and 12 failure modes. Evaluating ten contemporary models, we find that none is both competent and robust, that every model commits fake science on a substantial fraction of trapped tasks, and that most are miscalibrated skeptics—over-flagging clean data while still missing real flaws. We hope SciHarnessBench provides a durable target for measuring, and improving, the reliability of AI scientists.

A Per-Family Specification

Table 5 lists every v1 family: its identifier, the library that computes its ground truth, the trap it plants, whether it is a paired counterfactual twin or a robustness scenario, and the task.

B A Worked Clean/Trapped Pair

Consider `chem.theoretical_yield`. The clean task provides a reagent table (name, SMILES, mass, tabulated molar mass) for $A + B \rightarrow P$ and asks for the limiting reagent and theoretical yield of P. The trapped twin is byte-identical except that the *limiting* reagent's tabulated molar mass is scaled (e.g. by 1.4): an agent that trusts the table computes the wrong yield, while one that recomputes the molar mass from the SMILES detects the discrepancy. Detection is credited only for a structured issue of the form `{kind: decoy_data, evidence: {reagent: A, structure_molar_mass: 180.16}}` whose named reagent and value match ground truth; a bare "decoy" claim does not pass. A second family, `chem.reaction_energy`, presents the *same* barrier in kJ/mol (clean) versus kcal/mol (trapped) with an identical correct answer in kJ/mol; here detection requires naming the unit *and* reporting the correctly converted value, so guessing "kcal" without doing the conversion earns no credit. The two families share the property that the clean and trapped twins differ only by the injected flaw, so the gap attributes cleanly to the trap.

References

Benjamini, Y.; and Hochberg, Y. 1995. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1): 289–300.

Blyth, C. R. 1972. On Simpson's Paradox and the Sure-Thing Principle. *Journal of the American Statistical Association*, 67(338): 364–366.

Family	Library	Trap	Paired	Task
astro.aperture_photometry	astropy/scipy	corrupt input	yes	Aperture photometry with corrupt pixels
astro.brightness_threshold	astropy/scipy	unit mismatch	yes	Brightness threshold with unit mismatch
astro.transit_fit	astropy/scipy	non-convergence	yes	Transit-depth fit (convergence)
bio.diffexpr_confounding	Biopython/scipy	confounding	yes	Differential expression with batch confounding
bio.gc_content	Biopython/scipy	corrupt input	yes	Mean GC content with corrupt FASTA records
bio.wrong_control	Biopython/scipy	wrong control	yes	Treatment vs the tissue-matched control
chem.property_ranking	RDKit	corrupt input	yes	Rank by molecular weight with corrupt SMILES
chem.qm_convergence	RDKit	non-convergence	yes	Geometry optimization convergence
chem.reaction_energy	RDKit	unit mismatch	yes	Reaction feasibility with unit mismatch
chem.theoretical_yield	RDKit	decoy data	yes	Theoretical yield with decoy molar mass
climate.forecast_skill	numpy	future leakage	yes	Forecast skill with future leakage
climate.regional_threshold	numpy	unit mismatch	yes	Regional mean vs degC threshold with unit mismatch
climate.station_mean	numpy	decoy data	yes	Regional mean over stations with a decoy station
mat.alloy_bandgap	numpy	extrapolation	yes	Alloy bandgap extrapolation
mat.cell_density	numpy	corrupt input	yes	Densest crystal with corrupt structures
mat.formation_stability	numpy	unit mismatch	yes	Formation-energy stability with unit mismatch
neuro.channel_responsiveness	numpy/scipy	multiple comparisons	scenario	Channel responsiveness with multiple comparisons
neuro.decoding_circular	numpy/scipy	circular analysis	scenario	Neural decoding without double dipping
neuro.tuning_difference	numpy/scipy	underpowered	yes	Firing-rate difference from an adequate sample
phys.heat_cfl	numpy/scipy	non-convergence	yes	Heat-equation CFL stability
phys.kinetic_threshold	numpy/scipy	unit mismatch	yes	Kinetic-energy threshold with unit mismatch
phys.smallangle_extrapolation	numpy/scipy	extrapolation	yes	Small-angle fit extrapolation
stats.data_leakage	scipy/sklearn	data leakage	yes	Honest AUC with target leakage
stats.multiple_comparisons	scipy/sklearn	multiple comparisons	scenario	Multiple comparisons
stats.simpson	scipy/sklearn	confounding	yes	Simpson's paradox
stats.underpowered	scipy/sklearn	underpowered	yes	Underpowered conclusion

Table 5: The 26 task families of SciHarnessBench v1 (23 paired twins, 3 non-paired robustness scenarios).

Bragg, J.; D'Arcy, M.; Balepur, N.; Bareket, D.; Dalvi, B.; Feldman, S.; et al. 2026. AstaBench: Rigorous Benchmarking of AI Agents with a Scientific Research Suite. In *International Conference on Learning Representations (ICLR)*. Allen Institute for AI (Ai2).

Button, K. S.; Ioannidis, J. P. A.; Mokrysz, C.; Nosek, B. A.; Flint, J.; Robinson, E. S. J.; and Munafò, M. R. 2013. Power failure: why small sample size undermines the reliability of neuroscience. *Nature Reviews Neuroscience*, 14(5): 365–376.

Chan, J. S.; Chowdhury, N.; Jaffe, O.; Aung, J.; Sherburn, D.; Mays, E.; Starace, G.; Liu, K.; Maksin, L.; Patwardhan, T.; Weng, L.; and Madry, A. 2025. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. In *International Conference on Learning Representations (ICLR)*. OpenAI.

Chen, Z.; Chen, S.; Ning, Y.; Zhang, Q.; Wang, B.; Yu, B.; Li, Y.; Liao, Z.; Wei, C.; Lu, Z.; Dey, V.; Xue, M.; Baker, F. N.; Burns, B.; Adu-Ampratwum, D.; Huang, X.; Ning, X.; Gao, S.; Su, Y.; and Sun, H. 2025. ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery. In *International Conference on Learning Representations (ICLR)*.

Courant, R.; Friedrichs, K.; and Lewy, H. 1928. Über die partiellen Differenzengleichungen der mathematischen Physik. *Mathematische Annalen*, 100(1): 32–74. Original statement of the Courant–Friedrichs–Lewy (CFL) stability condition.

Gardner, M.; Artzi, Y.; Basmov, V.; Berant, J.; Bogin, B.; Chen, S.; Dasigi, P.; Dua, D.; Elazar, Y.; Gottumukkala, A.; et al. 2020. Evaluating Models' Local Decision Boundaries via Contrast Sets. In *Findings of the Association for Computational Linguistics: EMNLP 2020*.

Ioannidis, J. P. A. 2005. Why Most Published Research Findings Are False. *PLoS Medicine*, 2(8): e124.

Jansen, P.; Côté, M.-A.; Khot, T.; Bransom, E.; Mishra, B. D.; Majumder, B. P.; Tafjord, O.; and Clark, P. 2024. DiscoveryWorld: A Virtual Environment for Developing and Evaluating Automated Scientific Discovery Agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

Jimenez, C. E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; and Narasimhan, K. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations (ICLR)*.

Kapoor, S.; and Narayanan, A. 2023. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9): 100804.

Kriegeskorte, N.; Simmons, W. K.; Bellgowan, P. S. F.; and Baker, C. I. 2009. Circular analysis in systems neuroscience: the dangers of double dipping. *Nature Neuroscience*, 12(5): 535–540.

Laurent, J. M.; Janizek, J. D.; Ruzo, M.; Hinks, M. M.; Hammerling, M. J.; Narayanan, S.; Ponnapati, M.; White, A. D.; and Rodriques, S. G. 2024. LAB-Bench: Measuring Capabilities of Language Models for Biology Re-

search. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

Leek, J. T.; Scharpf, R. B.; Bravo, H. C.; Simcha, D.; Langmead, B.; Johnson, W. E.; Geman, D.; Baggerly, K.; and Irizarry, R. A. 2010. Tackling the widespread and critical impact of batch effects in high-throughput data. *Nature Reviews Genetics*, 11(10): 733–739.

Liang, P.; Bommasani, R.; Lee, T.; Tsipras, D.; Soylu, D.; Yasunaga, M.; Zhang, Y.; Narayanan, D.; Wu, Y.; Kumar, A.; et al. 2023. Holistic Evaluation of Language Models. *Transactions on Machine Learning Research (TMLR)*.

Mirza, A.; Alampara, N.; Kunchapu, S.; Ríos-García, M.; et al. 2025. A framework for evaluating the chemical knowledge and reasoning abilities of large language models against the expertise of chemists. *Nature Chemistry*, 17(7): 1027–1034. The ChemBench benchmark; arXiv:2404.01475, "Are large language models superhuman chemists?".

Mitchener, L.; Laurent, J. M.; Andonian, A.; Tenmann, B.; Narayanan, S.; Wellawatte, G. P.; et al. 2025. BixBench: a Comprehensive Benchmark for LLM-based Agents in Computational Biology. *arXiv preprint*. FutureHouse and ScienceMachine.

Notin, P.; Kollasch, A.; Ritter, D.; van Niekerk, L.; Paul, S.; Spinner, H.; Rollins, N.; Shaw, A.; Orenbuch, R.; Weitzman, R.; Frazer, J.; Dias, M.; Franceschi, D.; Gal, Y.; and Marks, D. 2023. ProteinGym: Large-Scale Benchmarks for Protein Fitness Prediction and Design. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. Preprint: bioRxiv 2023.12.07.570727.

Rasp, S.; Hoyer, S.; Merose, A.; Langmore, I.; Battaglia, P.; Russell, T.; Sanchez-Gonzalez, A.; Yang, V.; Carver, R.; Agrawal, S.; Chantry, M.; Bouallegue, Z. B.; Dueben, P.; Bromberg, C.; Sisk, J.; Barrington, L.; Bell, A.; and Sha, F. 2024. WeatherBench 2: A Benchmark for the Next Generation of Data-Driven Global Weather Models. *Journal of Advances in Modeling Earth Systems*, 16(6): e2023MS004019.

Rein, D.; Hou, B. L.; Stickland, A. C.; Petty, J.; Pang, R. Y.; Dirani, J.; Michael, J.; and Bowman, S. R. 2024. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. In *Conference on Language Modeling (COLM)*. First released on arXiv in 2023.

Ribeiro, M. T.; Wu, T.; Guestrin, C.; and Singh, S. 2020. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Riebesell, J.; Goodall, R. E. A.; Benner, P.; Chiang, Y.; Deng, B.; Lee, A. A.; Jain, A.; and Persson, K. A. 2025. Matbench Discovery – A framework to evaluate machine learning crystal stability predictions. *Nature Machine Intelligence*. First released on arXiv in 2023.

Segura, S.; Fraser, G.; Sanchez, A. B.; and Ruiz-Cortés, A. 2016. A Survey on Metamorphic Testing. *IEEE Transactions on Software Engineering*, 42(9): 805–824.

Simmons, J. P.; Nelson, L. D.; and Simonsohn, U. 2011. False-Positive Psychology: Undisclosed Flexibility in Data Collection and Analysis Allows Presenting Anything as Significant. *Psychological Science*, 22(11): 1359–1366.

Takamoto, M.; Praditia, T.; Leiteritz, R.; MacKinlay, D.; Alesiani, F.; Pflüger, D.; and Niepert, M. 2022. PDEBench: An Extensive Benchmark for Scientific Machine Learning. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

Tropsha, A. 2010. Best Practices for QSAR Model Development, Validation, and Exploitation. *Molecular Informatics*, 29(6-7): 476–488. Defines the applicability domain for QSAR extrapolation.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E. H.; Le, Q. V.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Wu, Z.; Ramsundar, B.; Feinberg, E. N.; Gomes, J.; Geniesse, C.; Pappu, A. S.; Leswing, K.; and Pande, V. 2018. MoleculeNet: a benchmark for molecular machine learning. *Chemical Science*, 9(2): 513–530.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

Zhang, Y.; Chen, X.; Jin, B.; Wang, S.; Ji, S.; Wang, W.; and Han, J. 2024. A Comprehensive Survey of Scientific Large Language Models and Their Applications in Scientific Discovery. *arXiv preprint*.